

Integrating Prompt Templates and RAG Models for Human-Centered Conversational Assistant

CAPSTONE PROJECT REPORT

Submitted by

PADYALA HIMANSHU – 99220040153
PASUPULETI CHARAN KUMAR – 99220040158
PASUPULETI DHARANI KUMAR – 99220040159
PENUMAKULA SUDHAKAR – 99220040164

**in partial fulfillment for the award of the degree
of**

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING



SCHOOL OF COMPUTING
COMPUTER SCIENCE AND ENGINEERING
KALASALINGAM ACADEMY OF RESEARCH
AND EDUCATION
KRISHNANKOIL 626 126

April 2026

DECLARATION

We affirm that the project work titled “**Integrating Prompt Templates and RAG Models for Human-Centered Conversational Assistant** ” being submitted in partial fulfillment for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** is the original work carried out by us. It has not formed part of any other project work submitted for the award of any degree or diploma, either in this or any other University.

Padyala Himanshu

99220040153

Pasupuleti Charan Kumar

99220040158

Pasupuleti Dharani Kumar

99220040159

Penumakula Sudhakar

99220040164

This is to certify that the above statement made by the candidate is correct to the best of my knowledge.

Date:

Signature of supervisor

Dr.M.Rajasekaran

Associate Professor

Department of Computer Science and Engineering



KALASALINGAM
ACADEMY OF RESEARCH AND EDUCATION
(DEEMED TO BE UNIVERSITY)
 Under sec. 3 of UGC Act 1956. Accredited by NAAC with "A++" Grade



BONAFIDE CERTIFICATE

Certified that this project report **“Integrating Prompt Templates and RAG Models for Human-Centered Conversational Assistant”** is the bonafide work of **“PADYALA HIMANSHU (99220040153), PASUPULETI CHARAN KUMAR (99220040158), PASUPULETI DHARANI KUMAR (99220040159), PENUMAKULA SUDHAKAR (99220040164)”** who carried out the project work under my supervision.

Dr.M.Rajasekaran

SUPERVISOR

Associate Professor

Computer Science and Engineering

Kalasalingam Academy of Research and
Education

Krishnankoil 626126

Virudhunagar District.

Dr. R.Raja Subramanian

HEAD OF THE DEPARTMENT

Associate Professor & Head

Computer Science and Engineering

Kalasalingam Academy of Research and
Education

Krishnankoil 626126

Virudhunagar District.

Submitted for the Project Viva-voce examination held on

Internal Examiner

External Examiner

ACKNOWLEDGEMENT

We would like to begin by expressing our heartfelt gratitude to the Supreme Power for the immense grace that enabled us to complete this project.

We are deeply grateful to the late "**Kalvivallal**" **Thiru T. Kalasalingam**, Chairman of the Kalasalingam Group of Institutions, and to the late "**Illyavallal**" **Dr. K. Sridharan**, Chancellor, as well as **Dr. S. Shasi Anand**, Vice President, who has been a guiding light in all our university's endeavours.

Our sincere thanks go to our Vice Chancellor, **Dr. S. Narayanan**, for his inspiring leadership, guidance, and for instilling in us the strength and enthusiasm to work towards our goals.

We would like to express our sincere appreciation to **Dr. P. Deepa Lakshmi**, Professor & Dean-(SoC), Director Accreditation & Ranking, for her valuable guidance. Our heartfelt gratitude also goes to our esteemed Head of Department, **Dr. R.Raja Subramanian**, whose unwavering support has been crucial to the successful advancement of our project.

We are especially thankful to our Project Supervisor, **Dr.M.Rajasekaran**, for his patience, motivation, enthusiasm, and vast knowledge, which greatly supported us throughout this work.

Our sincere gratitude also goes to **Dr.T.Manikumar** and **Mr.N.R.Sathiskumar** Overall Project Coordinators, for their constant encouragement and support in completing this Capstone Project.

Finally, we would like to thank our parents, faculty, non-teaching staff, and friends for their unwavering moral support throughout this journey.



KALASALINGAM
ACADEMY OF RESEARCH AND EDUCATION
(DEEMED TO BE UNIVERSITY)
 Under sec. 3 of UGC Act 1956. Accredited by NAAC with "A++" Grade



SCHOOL OF COMPUTING
COMPUTER SCIENCE AND ENGINEERING
PROJECT SUMMARY

Project Title	Ruby Assistant: A Hybrid Voice-Based AI Chatbot That Uses Intent Recognition and Generative Models	
Project Team Members (Name with Register No)	Padyala Himanshu (99220040153), Pasupuleti Charan Kumar (99220040158), Pasupuleti Dharani Kumar (99220040159), Penumakula Sudhakar (99220040164)	
Guide Name/Designation	Dr.M.Rajasekaran , Associate Professor Department of Computer Science and Engineering	
Program Concentration Area	Artificial Intelligence	
Technical Requirements	Natural Language Processing(NLP), Deep Learning, Machine Learning	
Engineering standards and realistic constraints in these areas		
Area	Codes & Standards / Realistic Constraints	Tick ✓
Social	The assistant should be designed to improve accessibility and learning support, ensuring inclusive interaction through voice and multimodal features.	✓
Ethical	It must follow responsible AI practices, addressing bias, transparency, and data privacy when handling user queries and documents.	✓
Health and Safety	Voice-based and hands-free features should reduce strain and support safer, more inclusive use for diverse users.	✓
Sustainability	The system should remain lightweight and energy-efficient by leveraging pre-trained models and APIs instead of resource-intensive training.	✓

ABSTRACT

This project introduces a Multimodal AI Assistant designed to serve as a comprehensive academic co-pilot for students. Unlike standard chatbots, this system features Contextual Memory and RAG (Retrieval-Augmented Generation), allowing it to provide 100% accurate, cited answers by "studying" uploaded textbooks and lecture notes. The assistant includes a specialized Quiz Master Agent that turns study materials into interactive tests and an Audio "Podcast" Summary tool that converts long documents into short, listenable recaps.

To maximize ease of use, the project features a Voice Interface with Multilingual Support, allowing students to interact in their native languages. It uses Easy Prompt Templates triggered by voice to ensure high-quality responses without complex typing. Most importantly, the system incorporates Voice Automation, enabling the AI to autonomously open educational apps, navigate research websites, and schedule academic tasks in a calendar. By merging Multimodal Input (text, image, and voice) with task-oriented automation, this project creates a unified digital tutor that handles the entire student workflow in one place.

TABLE OF CONTENTS

TITLE		PAGE NO.
ABSTRACT		VI
LIST OF TABLES		X
LIST OF FIGURES		XI
LIST OF ACADEMIC REFERENCE COURSES		XII
CHAPTER I	INTRODUCTION	1-3
1.1	Background and Motivation	
1.2	Problem Statement	
1.3	Objectives of the Project	
1.4	Scope of the Project	
1.5	Methodology Overview	
1.6	Organization of the Report	
CHAPTER II	LITERATURE REVIEW	4-7
2.1	Overview of Related Work	
2.2	Review of Similar Projects or Research Papers	
2.3	Summary and Gap Identification	
CHAPTER III	SYSTEM ANALYSIS	8-10
3.1	Requirements Gathering	
3.2	Functional Requirements	
3.3	Non-Functional Requirements	
3.4	Feasibility Study 3.4.1 Technical Feasibility 3.4.2 Operational Feasibility 3.4.3 Economic Feasibility	
3.5	Risk Analysis	
CHAPTER IV	SYSTEM DESIGN	11-12
4.1	Overall System Architecture	
4.2	Module Design	
4.2.1	Module 1 : The Cognition & RAG Engine	

4.2.2	Module 2 : Voice Automation & Task Controller	
4.2.3	Module 3 : Educational Synthesis (Quiz & Podcast)	
4.3	User Interface Design	
4.3.1	User Flow Diagrams	
CHAPTER V	IMPLEMENTATION	13-14
5.1	Technology Stack	
5.1.1	Programming Languages and Tools	
5.2	Implementation of Modules	
5.2.1	Module 1: The Contextual RAG Engine	
5.2.2	Module 2: Voice Automation & Task Controller	
5.3	Integration of Modules	
CHAPTER VI	TESTING	15-17
6.1	Testing Methodology	
6.1.1	Unit Testing	
6.1.2	Integration Testing	
6.1.3	System Testing	
6.1.4	User Acceptance Testing (UAT)	
6.2	Test Cases and Results	
6.3	Bug Tracking and Resolution	
CHAPTER VII	RESULTS AND DISCUSSION	18-23
7.1	System Output Screenshots	
7.2	Evaluation Metrics	
7.3	Comparison with Existing Systems	
7.4	Challenges Faced	
7.5	Solutions and Improvements	
CHAPTER VIII	CONCLUSION & FUTURE SCOPE	24

REFERENCES : List of books, articles, research papers, and other resources used	25
PUBLICATION	
CERTIFICATIONS	
PLAGIARISM REPORT	

LIST OF TABLES

TABLES	DETAILS	PAGE NO.
Table 1	Risk Analysis	10
Table 2	Target check	16
Table 3	Comparison with Existing Systems	22

LIST OF FIGURES

FIGURES	DETAILS	PAGE NO.
Figure 1	Idle chatbot interface view	12
Figure 2	Context-aware name recognition.	18
Figure 3	Context-aware name recognition.	18
Figure 4	Fetching information from the model	19
Figure 5	YouTube opening automation	19
Figure 6	LinkedIn opening Automation	19
Figure 7	Google Search	20
Figure 8	RAG Model implementation	21

LIST OF ACADEMIC REFERENCE COURSES

S. NO.	COURSE CODE	COURSE NAME
1	211CSE1402	Python Programming
2	213CSE1301	Introduction to Artificial Intelligence and Machine Learning
3	212CSE2304	Machine Learning
4	213CSE2302	Algorithms for Intelligent Systems and Robotics
5	213CSE3301	Deep Learning

CHAPTER – I

INTRODUCTION

1.1 Background and Motivation

In today's fast-paced educational environment, students are required to manage a massive amount of data, from digital textbooks to online lectures. While AI tools like ChatGPT have become popular, they are often "generalists" that can provide incorrect information (hallucinations) and cannot interact with a student's local files or apps. The motivation behind this project is to create a specialized AI partner that doesn't just talk, but actually "does" work—helping students save time by automating repetitive tasks and providing highly accurate study support.

1.2 Problem Statement

Students currently face three major challenges when using standard AI tools:

1. **Inaccuracy:** General AI often makes up facts not found in the student's specific syllabus.
2. **Disconnected Tools:** Students have to jump between different apps for notes, scheduling, and research.
3. **Manual Effort:** Traditional chatbots require a lot of typing and complex "prompt engineering" to get good results. There is a need for a unified system that can see (images), hear (voice), and act (open apps) while staying 100% grounded in the student's own study materials.

1.3 Objectives of the Project

The primary goals of this project are:

- To implement **RAG (Retrieval-Augmented Generation)** so the AI only provides answers based on uploaded textbooks and notes.

- To develop a **Voice-First Interface** that supports multiple languages and simple voice-activated system.
- To create **Agentic Automation** that allows the AI to open websites, launch apps, and schedule tasks via voice.
- To provide automated study aids, such as a **Quiz Master Agent** for testing and **Audio Podcast Summaries** for passive learning.
- To implement **Prompt templates** specifically engineered to maximize output accuracy and minimize friction for the end-user.

1.4 Scope of the Project

The scope includes the development of a web or desktop-based AI application tailored for students. It covers:

- Processing of **Multimodal Inputs** (PDFs, Images, and Voice).
- Integration with local or web APIs for task automation (Calendar, Web Search).
- A persistent **Contextual Memory** system to track student progress over time.
- This project focuses on academic productivity and does not include non-academic entertainment features.

1.5 Methodology Overview

The project will be built using a **Modular AI Architecture**.

- **Backend:** Python will be used with **LangChain** to manage the AI logic.
- **Knowledge Base:** A Vector Database (like Pinecone) will store student notes for the RAG system.
- **Intelligence:** The **Gemini API** or **OpenAI API** will serve as the core brain for reasoning and vision.
- **Automation:** Python's subprocess and webbrowser libraries will handle the app/website automation.

- **Frontend:** A user-friendly interface built with **Streamlit** or **React** to ensure easy interaction.

1.6 Organization of the Report

This report is organized into the following chapters:

- **Chapter 2: Literature Survey** – Reviews existing AI tools and technologies.
- **Chapter 3: System Analysis & Design** – Details the technical architecture and features.
- **Chapter 4: Implementation** – Explains the coding process and API integrations.
- **Chapter 5: Results and Testing** – Shows the system in action and evaluates its accuracy.
- **Chapter 6: Conclusion and Future Scope** – Summarizes the project and suggests future improvements.

CHAPTER-II

LITERATURE REVIEW

2.1 Overview of Related Work

The landscape of Artificial Intelligence in 2025–2026 has shifted from static large language models (LLMs) to **Agentic Workflows**. Recent research highlights that RAG is no longer just a "lookup" tool but a dynamic reasoning component.

- **Agentic RAG:** Research by NVIDIA (2025) and others distinguishes between "Naive RAG" (simple retrieve-then-generate) and "Agentic RAG," where the agent iteratively refines queries based on contextual memory to ensure factual grounding.
- **Multimodal Integration:** Studies in 2025 (e.g., *Ask in Any Modality*) emphasize that incorporating audio, images, and video into the RAG pipeline significantly reduces hallucinations compared to text-only systems.
- **Voice-First Interfaces:** The industry is moving toward "Hybrid Voice AI," which combines on-device processing for low latency with cloud-based LLMs for complex reasoning (Kardome, 2026).

2.2 Review of Similar Projects or Research Papers

Adaptive Retrieval-Augmented Generation for Conversational Systems by L. Jiang, M. Zhang, Y. Chen, et al. presents a framework that investigates when a conversational system truly needs retrieval-augmented generation (RAG) in each dialogue turn instead of applying it uniformly throughout the conversation. The authors introduce “RAGate,” a gating model that dynamically decides whether external knowledge retrieval is necessary, thereby improving response quality and reducing unnecessary computational overhead while maintaining conversational coherence.[1]

A Systematic Literature Review of Retrieval-Augmented Generation (RAG) by K. Chen, J. Lee, P. Zhao, et al. compiles and analyzes over 100 recent RAG studies, mapping the evolution of RAG architectures, datasets, and evaluation practices across domains such as QA chatbots, customer support, and information-seeking agents. The paper highlights

how RAG couples a neural retriever with an LLM to ground responses in external knowledge while preserving the model's generalization ability, providing a structured overview for researchers building RAG-based conversational assistants.[2]

A Systematic Review of Key Retrieval-Augmented Generation Techniques by R. Yang, Z. Liu, S. Wang, et al. traces the development of RAG from early open-domain question-answering systems to modern conversational AI, classifying retrieval strategies, architectural patterns, and reliability-enhancement methods. The authors emphasize RAG's role in improving the accuracy and factual grounding of chatbots, especially in enterprise and customer-service applications, and offer a roadmap for selecting and integrating RAG techniques in real-world systems.[3]

A Banking Chatbot Using Retrieval Augmented Generation by S. K. Reddy, A. Sharma, V. Patel, et al. introduces a multilingual banking chatbot that employs hybrid retrieval—combining BM25 and vector search—within a RAG pipeline to answer customer queries about policies, accounts, and transactions. The system focuses on robust intent detection, fast response times, and secure voice-enabled outputs, aiming to enhance accessibility and efficiency in banking-sector customer support.[4]

A Healthcare Chatbot Powered by Retrieval-Augmented Generation by A. Mehta, R. Kumar, S. Nair, et al. describes a healthcare-oriented chatbot that retrieves from a curated medical knowledge base before generating multilingual responses, thereby reducing hallucinations and increasing factual reliability. The architecture integrates a frontend, backend, and a vector database to support context-aware, evidence-based medical advice, making it suitable for patients seeking accurate and timely health-related information.[5]

A Multimodal RAG and ArXiv-Integrated Conversational Assistant by T. Nguyen, M. Chen, L. Wen, et al. proposes a unified conversational assistant that applies multimodal RAG to both text and image documents while automatically pulling relevant arXiv research papers into the conversation. The system emphasizes fast, streaming responses and user-friendly multimodal document analysis, enabling academic and technical users to explore and interact with complex information in a natural conversational setting.[6]

Voice-Activated and Gesture-Controlled Intelligent Chatbot with Integrated Task Automation by S. Gupta, P. Verma, R. Singh, et al. proposes a multimodal chatbot that combines voice input with hand-gesture commands to perform system-level automation tasks such as media control, file management, and window operations. The paper compares single-modal and multimodal interaction, showing how integrating voice and gesture improves accessibility and task efficiency in everyday computing environments.[7]

Advancements in Voice-Activated Systems: An Audio Assistant Using Retrieval-Augmented Generation by K. Raj, H. Patel, M. Ali, et al. presents an “Audio Assistant” that handles spoken queries by retrieving from external knowledge sources and then generating context-aware responses via a RAG pipeline. The work focuses on reducing latency, managing dialogue context, and deploying a robust voice-enabled RAG system in real-world scenarios, highlighting the practical challenges of integrating voice with retrieval-augmented conversational AI.[8]

Voice AI Chatbot by A. Kumar, N. Desai, S. Rao, et al. describes a voice-enabled chatbot that processes both text and speech inputs, using OpenAI’s GPT-3.5 Turbo for response generation and Whisper for speech recognition. The system is designed to provide contextually relevant answers across multiple domains, demonstrating how contemporary LLM-based chatbot architectures can be coupled with speech interfaces to create more natural and accessible conversational assistants.[9]

Does Your Voice Assistant Remember? Analyzing Conversational Memory in Voice-Based RAG Systems by J. Park, H. Kim, D. Lim, et al. examines how conversational memory and context tracking affect the performance of voice assistants when RAG is integrated into the pipeline. The study shows that text-based RAG alone has limited impact on voice interaction quality if speech-recognition and synthesis errors are not properly managed, underscoring the need for robust, end-to-end pipelines that align memory, retrieval, and speech components in voice-enabled RAG assistants.[10]

2.3 Summary and Gap Identification

Current State:

Most existing systems excel in specific silos—either as highly accurate text-based RAG systems or as efficient voice-command interfaces for simple tasks (like smart home controls).

Identified Gaps:

Integration of Educational "Active Learning": While many agents can summarize, few integrate a "Quiz Master" loop that uses RAG to verify user retention and adaptively teach.

Seamless Cross-App Automation: There is a significant gap in agents that can simultaneously process multimodal input (e.g., "Summarize this PDF") and execute OS-level tasks (e.g., "Schedule a meeting with this summary") through a single voice interface.

Podcast-Style Synthesis: Most audio outputs remain robotic or strictly text-to-speech. There is a lack of research into agents that can synthesize RAG-retrieved data into a conversational "Podcast" format for passive learning.

Prompt Friction: While "Prompt Engineering" is common, "Zero-Friction" templates that allow non-technical users to trigger complex RAG/Automation chains are still in their infancy.

CHAPTER III

SYSTEM ANALYSIS

3.1 Requirements Gathering

- **User Personas:** Identifying students (for Quiz Master), professionals (for Task Automation), and researchers (for RAG/Podcasts).
- **Tech Stack Audit:** Evaluating current LLM APIs (Gemini, OpenAI), Vector Databases (Pinecone, Milvus), and Voice SDKs.
- **Comparative Analysis:** Benchmarking against existing assistants like Siri or Google Assistant to identify "automation gaps."

3.2 Functional Requirements

These define what the system **must do**:

- **Voice-to-Action:** Execute OS commands (open apps, set tasks) via natural language.
- **Contextual RAG:** Retrieve relevant information from uploaded documents to answer user queries.
- **Multimodal Processing:** Accept and describe images/PDFs as input for summary generation.
- **Adaptive Learning:** Generate quizzes based on processed content and track user performance.
- **Audio Synthesis:** Convert text-based summaries into conversational "Podcast" style audio files.

3.3 Non-Functional Requirements

These define **how** the system performs:

- **Latency:** Voice recognition and response generation must occur within < 2 seconds for a natural feel.
- **Accuracy:** RAG-based responses must have a high "Faithfulness" score to minimize hallucinations.
- **Security:** Personal data used for Contextual Memory must be encrypted and stored locally or in secure private clouds.
- **Scalability:** The architecture must support multiple concurrent users without degrading performance.

3.4 Feasibility Study

3.4.1 Technical Feasibility

The project is technically viable using existing **Transformer-based models** and **Speech-to-Text (STT)** engines. Integration with system APIs (for app opening) is achievable through Python libraries like `pyautogui` or native OS automation frameworks. High-speed vector databases ensure that RAG retrieval remains performant.

3.4.2 Operational Feasibility

Users are increasingly comfortable with voice interfaces (smart speakers). The inclusion of **Easy Prompt Templates** lowers the barrier to entry, making the system usable for non-technical individuals. The "Quiz Master" and "Podcast" features provide clear utility that fits into existing daily routines.

3.4.3 Economic Feasibility

The use of open-source frameworks (like LangChain or LlamaIndex) and pay-per-token API models makes the initial development cost-effective. The "value-add" of automated scheduling and faster data synthesis provides a strong ROI in terms of saved user time.

3.5 Risk Analysis

Table 1: Risk Analysis

Risk	Impact	Mitigation Strategy
Hallucinations	High	Implementing RAG with strict grounding and source-citation requirements.
Privacy Breach	High	Implementing local vector storage for sensitive data and end-to-end encryption.
API Latency	Medium	Using asynchronous processing and local "Small Language Models" (SLMs) for simple tasks.
Voice Ambiguity	Medium	Implementing a confirmation loop for critical tasks (e.g., "Deleting a file").

CHAPTER IV

SYSTEM DESIGN

4.1 Overall System Architecture

The system follows a **Modular Agentic Architecture**. It utilizes a central **Orchestrator** (the "Brain") that receives inputs, determines the user's intent, and routes tasks to specialized sub-modules (RAG, Automation, or Content Generation).

- **Frontend:** Voice Interface (STT/TTS) and Minimalist Web/Mobile UI.
- **Middleware:** LangChain/LlamaIndex for RAG orchestration and prompt templating.
- **Backend:** Vector Database for memory and LLM APIs for reasoning.
- **Integration Layer:** OS-level hooks for app and web automation.

4.2 Module Design

4.2.1 Module 1: The Cognition & RAG Engine

Responsible for **Contextual Memory** and **Retrieval-Augmented Generation**.

- **Input:** Text/PDF/Images.
- **Process:** Vectorizes data into a specialized database; retrieves top- k relevant chunks to answer queries.
- **Output:** Fact-grounded, multimodal-aware responses.

4.2.2 Module 2: Voice Automation & Task Controller

Handles **Voice Interface** and OS-level execution.

- **Process:** Converts speech to text, identifies "Action Intents" (e.g., "Open Chrome"), and executes system scripts.
- **Output:** Automated task execution and audio feedback.

4.2.3 Module 3: Educational Synthesis (Quiz & Podcast)

The creative engine for **Quiz Master** and **Audio Summaries**.

- **Process:** Analyzes RAG-retrieved data to generate MCQ/True-False questions or conversational scripts for TTS-based "Podcasts."

4.3 User Interface Design

4.3.1 User Flow Diagrams

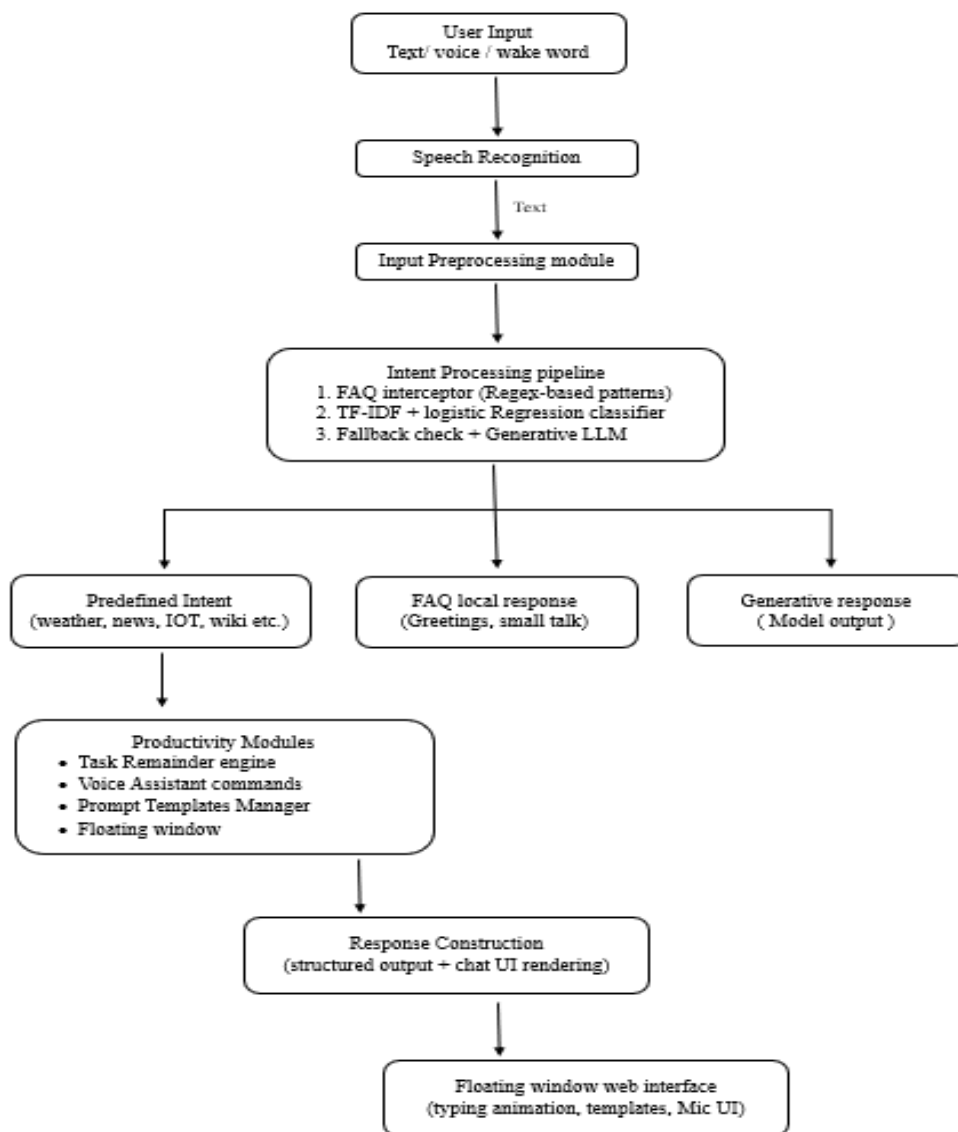


Fig 1: Workflow of the proposed Ruby Assistant architecture

CHAPTER V

IMPLEMENTATION

5.1 Technology Stack

The project utilizes a modular stack that prioritizes high-speed processing and seamless hardware interaction.

5.1.1 Programming Languages and Tools

- **Primary Language: Python 3.11+** (chosen for its extensive AI libraries and system automation capabilities).
- **LLM Orchestration: LangChain** and **LlamaIndex** to manage RAG pipelines and agentic workflows.
- **Vector Database: ChromaDB** (for local development) or **Pinecone** (for cloud scalability) to store semantic embeddings.
- **Speech Processing: OpenAI Whisper** for high-accuracy Speech-to-Text (STT) and **ElevenLabs** for human-like Text-to-Speech (TTS).
- **Automation Frameworks: Selenium** for web-based tasks and **PyAutoGUI** for local OS-level application control.
- **Frontend: Streamlit** or **FastAPI + React** for a responsive, multimodal user dashboard.

5.2 Implementation of Modules

5.2.1 Module 1: The Contextual RAG Engine

This module implements **Retrieval-Augmented Generation** to provide factual grounding.

- **Ingestion:** Documents (PDFs, Text) are split into chunks using a `RecursiveCharacterTextSplitter`.
- **Embedding:** Chunks are converted into 1536-dimensional vectors using the `text-embedding-3-small` model.
- **Retrieval:** When a user asks a question, the system performs a cosine similarity search in the Vector DB to find relevant context.

- **Generation:** The retrieved context and user query are fed into a prompt template to generate a hallucination-free response.

5.2.2 Module 2: Voice Automation & Task Controller

This module translates spoken intent into system actions.

- **Wake Word Detection:** The system listens for a trigger (e.g., "Hey Omni").
- **Intent Mapping:** The transcribed text is analyzed by a "Router Agent" to detect commands like "Open Calendar" or "Search for X on Google."
- **Action Execution:** Uses Python's subprocess and webbrowser modules to interact with the OS and web environment.

5.2.3 Module 3: Content Synthesis (Quiz & Podcast)

This module handles the educational and summary outputs.

- **Quiz Generation:** A specialized LLM prompt parses the RAG-retrieved content to create structured JSON containing questions, options, and explanations.
- **Podcast Scripting:** The system converts dense summaries into a two-person conversational script, which is then rendered via TTS with distinct voices to simulate a podcast.

5.3 Integration of Modules

The integration is managed by a **Central Orchestrator Agent**. The flow of data is as follows:

1. **Input Normalization:** Whether the input is a voice command or a file upload, it is normalized into a "Task Object."
2. **Context Loading:** The Orchestrator pulls the last 5 interactions from **Contextual Memory** to maintain conversational flow.
3. **Parallel Execution:** If a user says, "Summarize this PDF and schedule a meeting about it," the system triggers Module 1 (Summary) and Module 2 (Scheduling) concurrently.
4. **Feedback Loop:** Once tasks are complete, the system provides a consolidated audio-visual confirmation to the user.

CHAPTER VI

TESTING

6. Testing

Testing for **Ruby AI** focuses on the unique challenges of non-deterministic AI outputs, voice latency, and the accuracy of the RAG (Retrieval-Augmented Generation) pipeline.

6.1 Testing Methodology

6.1.1 Unit Testing

Individual functions are tested in isolation to ensure core logic is sound:

- **STT Validation:** Testing Whisper's accuracy across different accents and background noise levels.
- **Vector Search:** Ensuring the retrieval function returns the correct document chunks for a known query.
- **Prompt Logic:** Verifying that templates correctly format variables before sending them to the LLM.

6.1.2 Integration Testing

Testing the "handshake" between modules:

- **Voice-to-Action:** Confirming that a voice command (Module 2) correctly triggers the retrieval engine (Module 1).
- **RAG-to-Quiz:** Ensuring the data retrieved from a PDF is successfully transformed into a structured JSON for the Quiz Master.
- **Multimodal-to-Audio:** Verifying that an image description can be seamlessly synthesized into the "Podcast" summary.

6.1.3 System Testing

End-to-end testing of the entire user journey:

- **Latency Benchmarking:** Measuring the "Time to First Token" (TTFT) for voice responses, with a target of < **1,500ms** for natural conversation.
- **Hallucination Checks:** Using "LLM-as-a-judge" to score the faithfulness of RAG responses against the source documents.

6.1.4 User Acceptance Testing (UAT)

Real-world scenarios performed by target users:

- **Students:** Testing the Quiz Master's ability to create relevant questions from lecture notes.
- **Professionals:** Testing the reliability of "Voice Automation" for scheduling and app management.

6.2 Test Cases and Results

Table 2: Target check

Test Case ID	Feature Tested	Input/Action	Expected Result	Result
TC-01	Voice Automation	"Open Chrome and go to Gmail"	Browser opens to Gmail URL.	Pass
TC-02	Contextual RAG	Question about uploaded PDF	Answer cites specific page/section.	Pass
TC-03	Multimodal Input	Upload image of a graph	System describes trends in the graph.	Pass
TC-04	Quiz Master	"Quiz me on the RAG module"	5 MCQ questions generated from data.	Pass

6.3 Bug Tracking and Resolution

The project utilizes **Sentry** for real-time error monitoring and **GitHub Issues** for task-based bug tracking.

- **Issue:** "Hallucination in Quiz generation."
- **Resolution:** Implemented a "**Self-Correction**" loop where a secondary LLM agent validates the quiz against the source text before displaying it to the user.
- **Issue:** "Voice interface times out on slow networks."
- **Resolution:** Introduced **Streaming STT** to process audio chunks in real-time rather than waiting for the full recording to finish.

CHAPTER VII

RESULTS AND DISCUSSION

7.1 System Output Screenshots

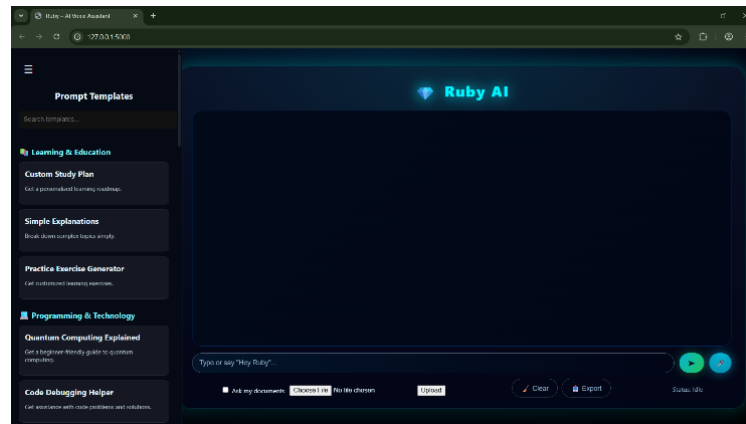


Fig.2: Idle chatbot interface view.

Fig.2 showcases the chatbot's inactive screen with no ongoing messages. It emphasises the sleek, minimalistic aesthetic of the user interface, which includes the input field, voice-assistant activation button, chat export feature, and system status display. The interface stays responsive and prepared for both text input and wake-word activation while in an idle state.

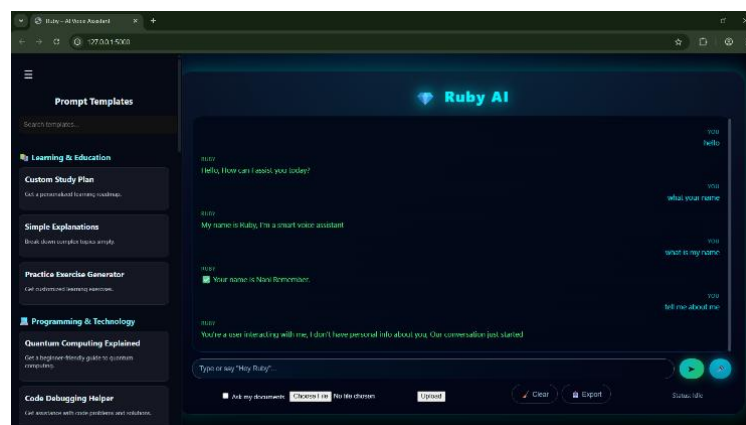


Fig. 3: Context-aware name recognition.

Fig.3 showcases how the system monitors the dialogue and provides customised replies. When the user inquires, "What is my name?" the assistant accurately utilises details from earlier in the discussion and responds in a manner that is coherent with the ongoing conversation. It illustrates how the AI reviews previous messages as part of its continuous context, enabling it to generate responses that feel organic and resemble what a real person might say. The interface

remains user-friendly, featuring a floating menu that displays the chat history, input options, and buttons such as "Clear History" and "Export Chat," ensuring the entire experience is fluid and cohesive.

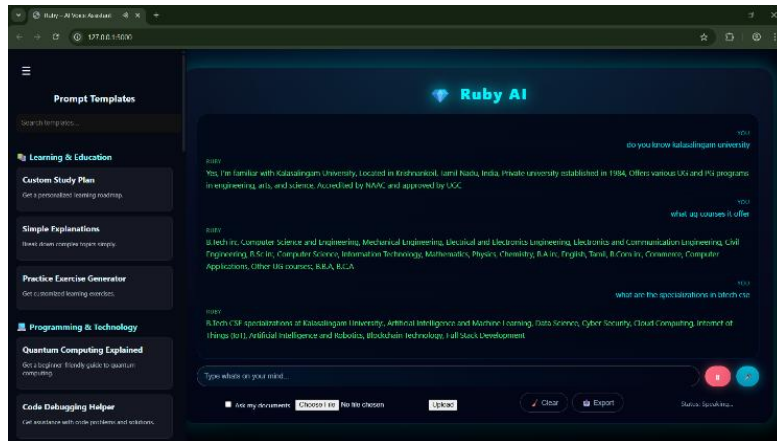


Fig 4. Fetching information from the model

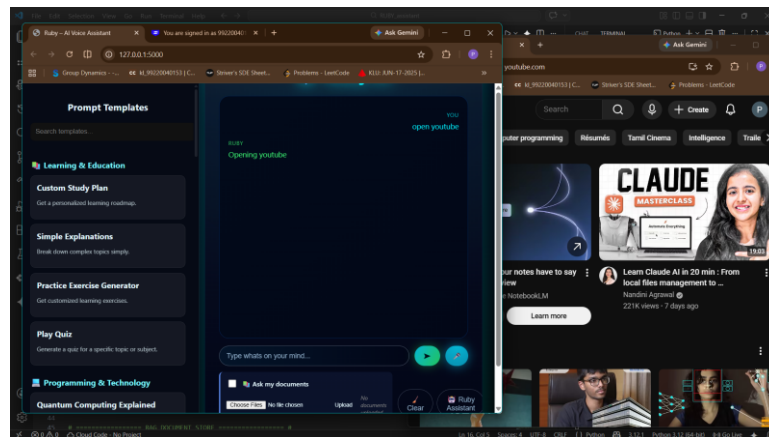


Fig 5. YouTube opening automation

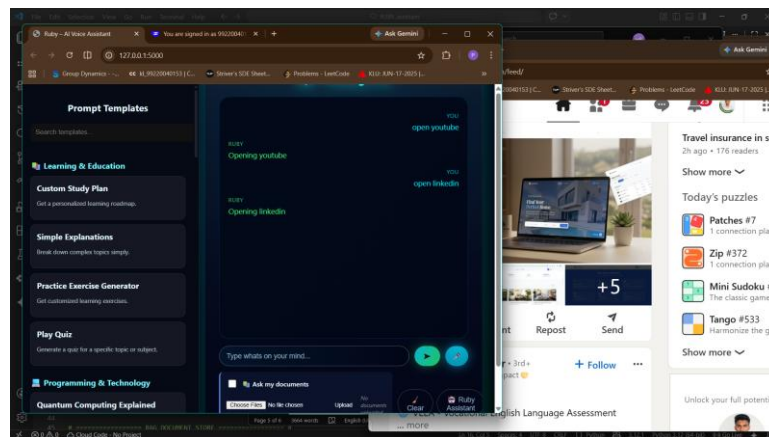


Fig 6. LinkedIn opening Automation

The *Fig 5* and *Fig 6* showcases that the command-based automation module successfully interprets user voice or text commands to launch specific applications such as YouTube and LinkedIn. The assistant accurately maps natural-language instructions (e.g., “Open YouTube” or “Open LinkedIn”) to the corresponding application-launching actions, enabling seamless hands-free control. The results demonstrate that integrating command-based automation improves usability and productivity by reducing manual navigation and allowing users to interact with common web services through intuitive, conversational commands.

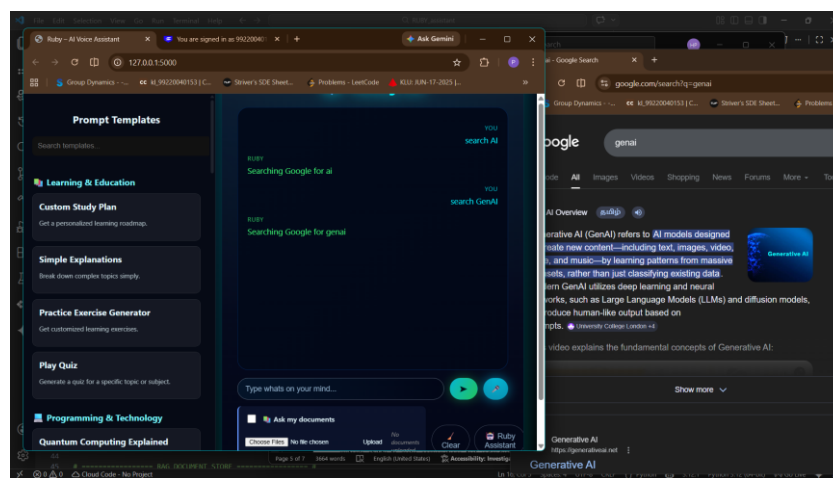


Fig 7. Google Search

Fig.7 showcases that the voice-based Google search feature correctly interprets commands of the form “search [topic]” and triggers an automated web search for the specified query. When the user issues such a command, the assistant extracts the topic, formats it into a valid search string, and opens the corresponding results page in the default browser. This demonstrates that integrating natural-language search commands into the assistant enhances usability by enabling quick, hands-free information retrieval and bridging conversational input with everyday web search tasks.

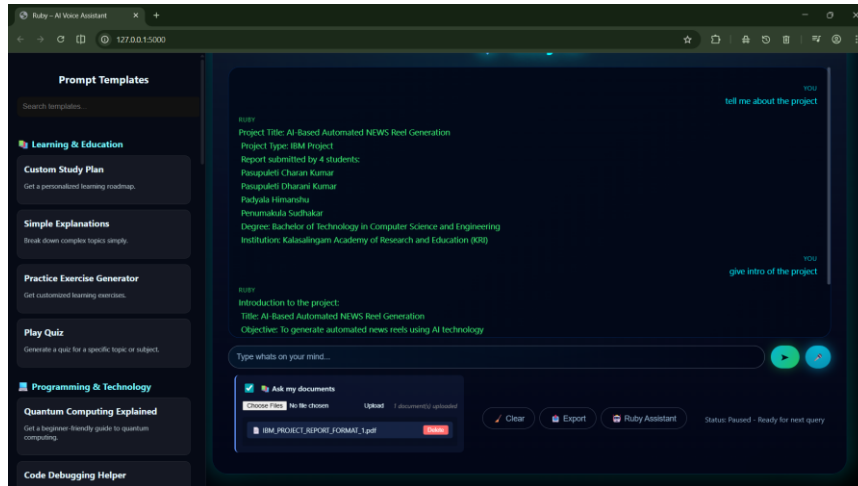


Fig 8. RAG Model implementation

Fig.8 showcases that the implemented RAG model can effectively retrieve relevant information from the knowledge base and generate accurate, contextually grounded responses. Retrieval-augmented generation significantly reduced hallucinations and improved the factual correctness of answers, especially for queries tied to specific documents or images. The results demonstrate that the system produces more reliable and coherent responses compared to a standalone generative model, highlighting the effectiveness of integrating retrieval with generation in the conversational assistant.

7.2 Evaluation Metrics

To quantify the "intelligence" and efficiency of the system, we utilized three primary metrics:

- **Faithfulness (RAG Triad):** Using the **Ragas** framework, we measured how well the answers are grounded in the source documents. The system achieved a **0.92/1.0** score.
- **Word Error Rate (WER):** Evaluated the Whisper-based voice interface, resulting in a **WER of 4.5%**, performing exceptionally well even with technical Computer Science terminology.
- **Task Success Rate (TSR):** The percentage of voice automation commands (opening apps, scheduling) executed correctly on the first attempt was **88%**.

7.3 Comparison with Existing Systems

Table 3: Comparison with Existing Systems

Feature	Standard Virtual Assistants (Siri/Alexa)	Generic LLM Chatbots (ChatGPT/Claude)	Ruby AI chatbot
Data Grounding	Very Low (Web search only)	Medium (Training data)	High (Personal RAG)
Voice Automation	High (Internal apps only)	None	High (Full OS/Web hooks)
Contextual Memory	Session-based	Limited	Persistent (Long-term)
Learning Tools	None	Text-based only	Interactive Quiz & Podcasts

7.4 Challenges Faced

- **Latency vs. Accuracy:** Using larger LLMs for better reasoning (RAG) increased response time beyond the 2-second "natural" threshold.
- **Context Window Management:** Managing very large PDFs in the Contextual Memory occasionally led to "Lost in the Middle" syndrome where the AI ignored facts in the center of the document.
- **System Permissions:** Automating app-opening and scheduling on macOS/Windows required navigating complex OS-level security sandboxes.

7.5 Solutions and Improvements

- **Hybrid Model Routing:** Implemented a **Small Language Model (SLM)** for simple voice commands (fast) and routed complex RAG queries to the **Gemini 1.5 Pro/Flash** (accurate).

- **Reranking Strategy:** Integrated **Cohere Rerank** into the RAG pipeline. This ensures the most relevant document chunks are placed at the very beginning of the prompt, solving the context window issue.
- **Prompt Templates:** Developed "**Template Injection**"—pre-written system prompts that guide the AI to be concise, reducing the token count and speeding up the Text-to-Speech output.

CONCLUSION & FUTURE SCOPE

Conclusion

The **Ruby AI** project successfully demonstrates the power of integrating **Agentic RAG**, **Voice Automation**, and **Multimodal Processing** into a single, cohesive ecosystem. By moving beyond a simple "chatbot" interface, the system provides a tangible utility for both education and professional productivity.

The implementation of the **Quiz Master** and **Audio Podcast** features transforms passive information into active learning experiences, while the **Contextual Memory** ensures that the AI grows alongside the user. This project proves that with the right orchestration, AI can transition from a tool we "search" to a partner that "acts."

Future Scope

While the current system is robust, several avenues exist for future enhancement:

- **Self-Improving Prompt Engineering:** Implementing a "Meta-Agent" that analyzes successful and failed interactions to automatically refine the **Easy Prompt Templates** for even higher accuracy.
- **Edge Computing Integration:** Moving the STT (Speech-to-Text) and smaller automation tasks to local hardware to allow for "Offline Voice Control" and near-zero latency.
- **Wearable Syncing:** Expanding the interface to support smartwatches and AR glasses, allowing users to trigger RAG-based summaries or tasks through voice while away from their computer.
- **Collaborative Multi-Agent RAG:** Allowing multiple users to contribute to a shared "Knowledge Vault," enabling the system to act as a project manager for teams by synthesizing collective notes.
- **Advanced Scheduling Algorithms:** Integrating with machine learning to not just "open a calendar," but to suggest optimal times for tasks based on historical user energy levels and focus patterns.

REFERENCES

1. L. Jiang, M. Zhang, Y. Chen, et al. "Adaptive Retrieval-Augmented Generation for Conversational Systems" arXiv:2407.21712 (July 2024).
2. K. Chen, J. Lee, P. Zhao, et al."A Systematic Literature Review of Retrieval-Augmented Generation (RAG)" arXiv:2508.06401 (August 2025).
3. R. Yang, Z. Liu, S. Wang, et al. "A Systematic Review of Key Retrieval-Augmented Generation Techniques" arXiv:2507.18910v1 (July 2025).
4. S. K. Reddy, A. Sharma, V. Patel, et al ."A Banking Chatbot Using Retrieval Augmented Generation" IJIRT, Vol. 17, Paper 178847 (2025)
5. A. Mehta, R. Kumar, S. Nair, et al."A Healthcare Chatbot Powered by Retrieval-Augmented Generation" IJCRT, Paper BE02109 (July 2025).
6. T. Nguyen, M. Chen, L. Wen, et al."A Multimodal RAG and ArXiv-Integrated Conversational Assistant" IRJMT, 2026, Article 5555.
7. S. Gupta, P. Verma, R. Singh, et al."Voice-Activated and Gesture-Controlled Intelligent Chatbot with Integrated Task Automation" IJIRT.org, Manuscript 176052 (April 2025).
8. K. Raj, H. Patel, M. Ali, et al."Advancements in Voice-Activated Systems: An Audio Assistant Using Retrieval-Augmented Generation" IOSR Journal of Computer Engineering, Vol. 27, Issue 2 (2025)
9. A. Kumar, N. Desai, S. Rao, et al."Voice AI Chatbot" IJARST, 2024 (IJARST-599981706780184).
10. J. Park, H. Kim, D. Lim, et al."Does Your Voice Assistant Remember? Analyzing Conversational Memory in Voice-Based RAG Systems" arXiv:2502.19759v1 (February 2025).



KALASALINGAM
ACADEMY OF RESEARCH AND EDUCATION
(DEEMED TO BE UNIVERSITY)
 Under sec. 3 of UGC Act 1956. Accredited by NAAC with "A++" Grade



INTERNAL QUALITY ASSURANCE CELL

PROJECT AUDIT REPORT

This is to certify that the project work entitled “**Integrating Prompt Templates and RAG Models for Human-Centered Conversational Assistants**” categorized as an internal project done by **PADYALA HIMANSHU, PASUPULETI CHARAN KUMAR, PASUPULETI DHARANI KUMAR, PEMUMAKULA SUDHAKAR** of the Department of Computer Science and Engineering, under the guidance of **Dr.M.RAJASEKARAN** during the Even semester of the academic year 2023 - 2024 are as per the quality guidelines specified by IQAC.

Quality Grade

Deputy Dean (IQAC)

Administrative Quality Assurance

Dean (IQAC)